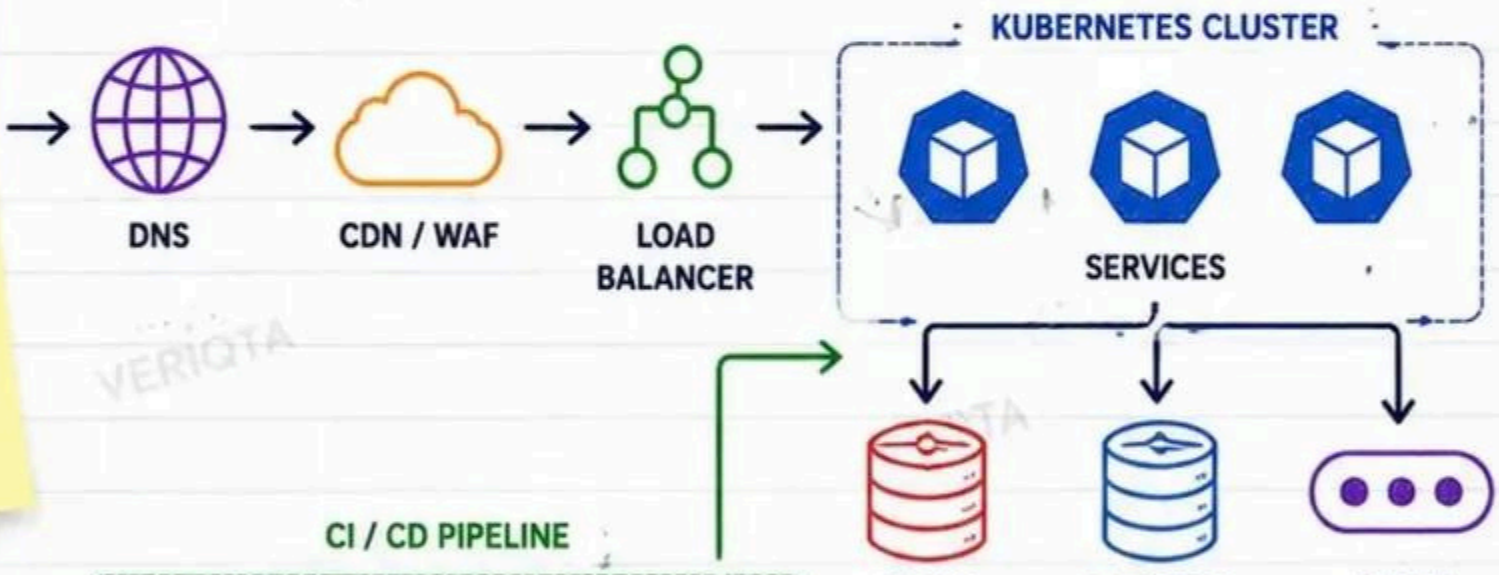
VERIQTA

DEVOPS SYSTEM DESIGN HANDBOOK

FROM ARCHITECTURE DECISIONS TO PRODUCTION-SCALE SYSTEMS

DESIGN SMARTER

- ✓ Make better architecture decisions
- ✓ Build systems that scale
- ✓ Operate with confidence



BUILT FOR ENGINEERS

- ✓ Real-world architectures
- ✓ Failure-aware design
- ✓ Production-ready thinking

COVERING

- ✓ Architecture Patterns
- ✓ Scaling
- ✓ Reliability
- ✓ CI/CD Design
- ✓ Observability
- ✓ Security
- ✓ Disaster Recovery

★ DESIGN • AUTOMATE • DEPLOY • OPERATE • SCALE ★

HOW SENIOR ENGINEERS APPROACH SYSTEM DESIGN

GREAT SYSTEMS START WITH CLARITY, NOT TECHNOLOGY

1 REQUIREMENTS BEFORE ARCHITECTURE

- Understand the problem deeply.
- Know what the system must do.
- Define success before design.

2 FUNCTIONAL VS NON-FUNCTIONAL REQUIREMENTS

- Functional: Features, actions, data, behavior.
- Non-functional: Quality attributes like performance, security, reliability.

3 QUALITY ATTRIBUTES

- Availability – system uptime and reliability.
- Latency – response time expectations.
- Throughput – requests per second the system must handle.
- Durability – data must not be lost.

4 RTO AND RPO

- RTO (Recovery Time Objective): How fast the system must recover.
- RPO (Recovery Point Objective): How much data loss is acceptable.

5 CAPACITY ASSUMPTIONS

- Expected users and growth.
- Peak loads and traffic patterns.
- Data volume and storage growth.
- Plan for scale, not yesterday.

6 IDENTIFYING CRITICAL PATHS

- Map the end-to-end user journey.
- Identify the steps that directly impact the user.
- Optimize what is critical, not what is convenient.

7 FINDING LIKELY FAILURE DOMAINS

- Components that can fail.
- External dependencies.
- Single points of failure.
- Plan for failure before it happens.

8 ARCHITECTURE TRADE-OFFS

- Every decision has a cost.
- Performance vs cost.
- Consistency vs availability.
- Simplicity vs flexibility.

PRODUCTION NOTE

NEVER START BY CHOOSING TOOLS

Tools come and go.
Good architecture lasts.
Solve the problem first, then choose the right tools for the solution.

DESIGN PROCESS (BEFORE TOOLS)

1. UNDERSTAND THE PROBLEM → 2. GATHER REQUIREMENTS → 3. ANALYZE & IDENTIFY CONSTRAINTS → 4. DESIGN SOLUTION → 5. VALIDATE & IMPROVE

★ **CLARITY FIRST. ARCHITECTURE SECOND. TOOLS LAST.**

DESIGNING A PRODUCTION WEB PLATFORM

USER → DNS → CDN → WAF → LOAD BALANCER → APPLICATION → DATABASE

PUBLIC & PRIVATE BOUNDARIES

- Keep internet-facing components in public subnets.
- Keep applications and databases in private subnets.

STATELESS APPLICATION DESIGN

- No server-side session storage.
- Easy to scale horizontally.
- Improved resilience.

SESSION MANAGEMENT

- Store sessions in a distributed store (Redis / DynamoDB).
- Use secure, short-lived tokens.

CACHING

- CDN for static content.
- Application cache for dynamic data.
- Database query caching.

DATABASE PLACEMENT

- Place databases in private subnets.
- Use Multi-AZ for high availability.
- Read replicas for scaling reads.

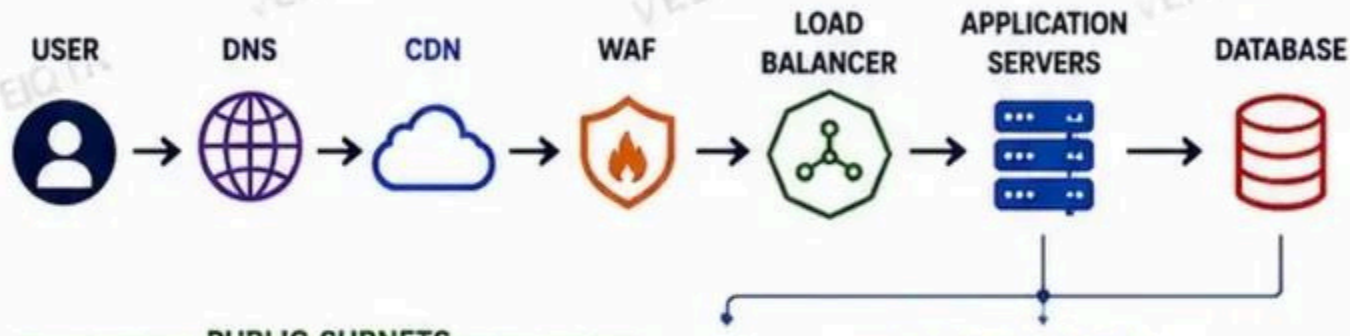
HORIZONTAL SCALING

- Scale application instances behind the load balancer.
- Use Auto Scaling based on load.

FAILURE POINTS

- DNS, CDN, WAF, Load Balancer, Application, Database, Network.
- Design for failure at every layer.

COMPLETE REQUEST LIFECYCLE



SECURE ISOLATION
Protect critical resources.

HIGH AVAILABILITY
Multi-AZ and redundancy.

PERFORMANCE
Caching and CDN reduce latency.

SCALABILITY
Scale horizontally based on demand.

RESILIENCE
Designed for failures and recovery.

REQUEST FLOW EXPLAINED

1. **USER REQUESTS THE DOMAIN** — User sends request from browser or mobile app.
2. **DNS RESOLVES THE DOMAIN** — DNS returns the CDN endpoint IP address.
3. **CDN SERVES CONTENT** — Static content served from edge closest to the user.
4. **WAF INSPECTS TRAFFIC** — Filters malicious requests and blocks threats.
5. **TRAFFIC ROUTED TO LOAD BALANCER** — Distributes traffic across healthy instances.
6. **APPLICATION PROCESSES REQUEST** — App handles business logic (stateless).
7. **DATABASE & CACHE ACCESS** — Reads/writes dat from DB and cac layers.

SENIOR ENGINEER TIP

DESIGN THE REQUEST PATH BEFORE SELECTING INFRASTRUCTURE OR TOOLS.

HIGHLY AVAILABLE MULTI-AZ ARCHITECTURE

DESIGNED FOR FAILURE. BUILT FOR CONTINUITY.

ELIMINATING SINGLE POINTS OF FAILURE

- No critical component should depend on a single AZ.
- Every layer is redundant.

LOAD BALANCER DISTRIBUTION

- Internet-facing ALB spans all AZs.
- Routes traffic only to healthy targets.

APPLICATION REPLICAS

- Run across multiple AZs.
- Auto Scaling replaces unhealthy instances.
- Stateless design enables easy failover.

DATABASE MULTI-AZ DESIGN

- Primary DB in one AZ.
- Synchronous standby in another AZ.
- Automatic failover with minimal downtime.

NAT & NETWORK DEPENDENCIES

- NAT Gateways in each public subnet.
- Route tables are AZ-aware.

AZ FAILURE SCENARIOS

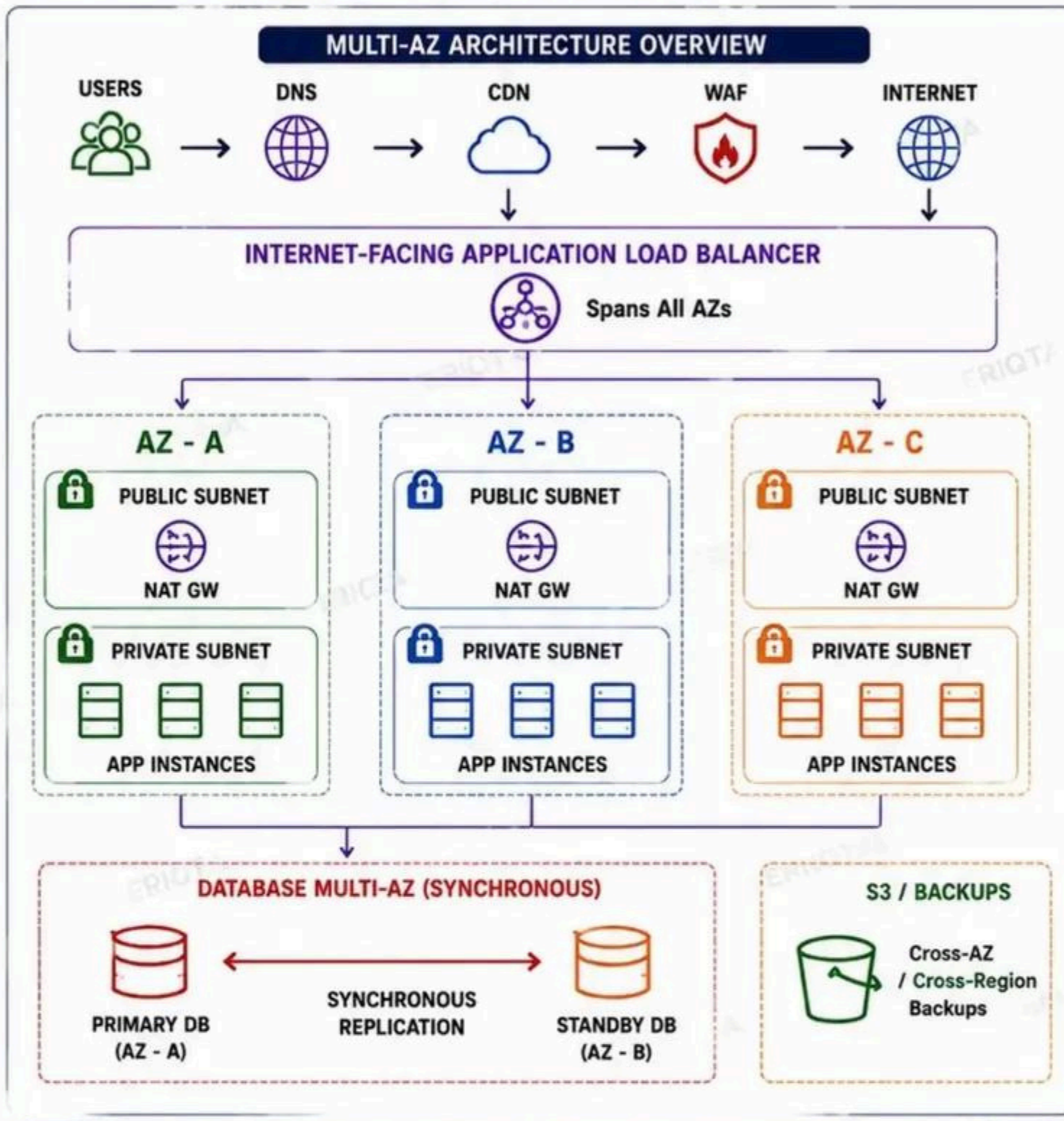
- Entire AZ can fail.
- Architecture keeps running in remaining AZs.

QUORUM CONSIDERATIONS

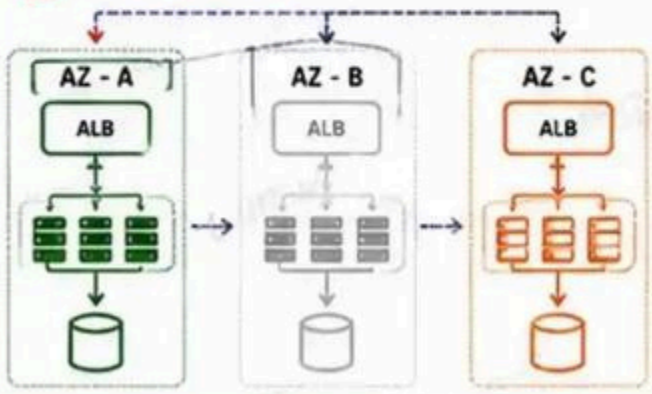
- Use odd number of nodes for quorum.
- Maintain majority during failures.

GRACEFUL DEGRADATION

- System continues with reduced capacity.
- Non-critical features degrade first.



AZ FAILURE SCENARIO: WHAT HAPPENS WHEN ONE AZ DISAPPEARS?



SYSTEM REMAINS AVAILABLE

- ALB routes traffic to healthy AZs (A & C).
- App instances in A & C continue serving traffic.
- Database automatically fails over to standby in B.
- Data remains consistent (synchronous replication).
- Users experience minimal or no downtime.

SENIOR ENGINEER TIP

DESIGN THE SYSTEM SO ANY SINGLE AZ CAN FAIL WITHOUT IMPACTING AVAILABILITY OR DATA.

KEY BENEFITS

HIGH AVAILABILITY · FAULT TOLERANCE · SCALABILITY · AUTOMATED RECOVERY · DATA DURABILITY

DESIGNING FOR MILLIONS OF REQUESTS

SCALE SMART. STAY FAST. STAY AVAILABLE.

1. REQUESTS PER SECOND
- Measure and plan for peak RPS.
- Account for traffic spikes.

2. CONCURRENT CONNECTIONS
- Plan for active users, not just requests.
- Control connection lifetime.

3. HORIZONTAL VS VERTICAL SCALING
- Vertical: More power per instance.
- Horizontal: More instances.
- Horizontal wins for scale and resilience.

4. AUTO SCALING
- Scale out on demand.
- Scale in to save cost.
- Use metrics, not guesswork.

5. LOAD BALANCING
- Distribute traffic evenly.
- Health checks keep traffic on healthy instances.

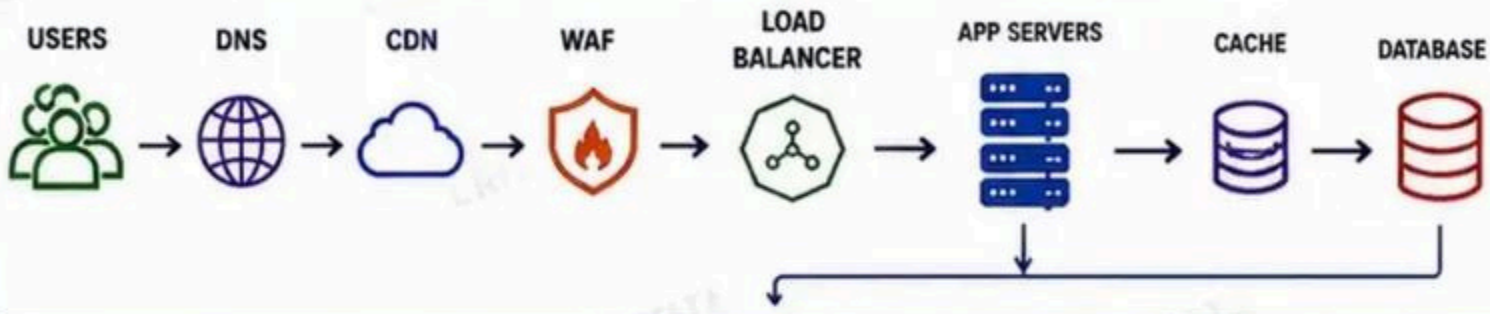
6. CONNECTION POOLING
- Reuse connections.
- Reduce latency and overhead.
- Prevent connection exhaustion.

7. CACHING LAYERS
- CDN, App Cache, Database Cache.
- Reduce load on backend systems.

8. DATABASE BOTTLENECKS
- DB cannot scale like stateless apps.
- Use replicas, sharding, caching, indexing.

9. BACKPRESSURE
- Protect the system when overloaded.
- Queue, shed, or slow down requests.

TYPICAL HIGH-SCALE REQUEST FLOW



SCALING STRATEGY LAYERS

TRAFFIC LAYER	DNS, CDN, WAF absorb and filter large volumes of traffic close to the user.
DISTRIBUTION LAYER	Load balancers distribute traffic across healthy instances across multiple AZs.
COMPUTE LAYER	Application instances scale horizontally behind the load balancer. Stateless by design.
CACHE LAYER	Multiple caching layers reduce load on compute and database.
DATA LAYER	Databases use replicas, partitioning, and caching to handle massive read/write loads.
RESILIENCY LAYER	Auto Scaling, Health Checks, Multi-AZ deployment, Backpressure, and Failover protect the system.

HORIZONTAL VS VERTICAL SCALING

VERTICAL SCALING	HORIZONTAL SCALING
Limited by hardware ceiling	Add more instances
Downtime required	High availability
Expensive at high scale	Cost efficient
Single point of failure	Elastic and resilient

SYSTEM PRESSURE HANDLING

MONITOR LOAD
Track RPS, Latency, Errors, CPU, Memory, Connections.

DETECT PRESSURE
Identify saturation before failures.

APPLY BACKPRESSURE
Queue requests, rate limit, or shed excess load.

PROTECT SYSTEM
Keep core functionality available for critical users.

COMMON BOTTLENECKS

CPU	MEMORY	DATABASE	NETWORK	DISK / IO
High CPU usage causes slow responses.	Out of memory leads to crashes and restarts.	Slow queries, locks, or maxed connections.	High latency or limited bandwidth reduces throughput.	Slow disk or high IO wait hurts performance.

QUICK CAPACITY FORMULA

$$\text{Required Instances} = \frac{\text{RPS} \times \text{Avg Response Time (s)}}{\text{Target Utilization} \times 3600}$$

SCALING NOTE

SCALING COMPUTE DOES NOT AUTOMATICALLY SCALE THE SYSTEM.

SENIOR ENGINEER TIP

DESIGN FOR THE REQUEST PATH, NOT FOR THE TOOL.

VERIQTA

LOAD BALANCING AND TRAFFIC ENGINEERING

DISTRIBUTE TRAFFIC. MAXIMIZE AVAILABILITY. PROTECT YOUR SYSTEM.

1 LAYER 4 VS LAYER 7

- Layer 4: Routes based on IP and port. Faster and simpler.
- Layer 7: Routes based on content (HTTP, path, host). Smarter and more flexible.

2 HEALTH CHECKS

- Continuously check the health of targets.
- Only send traffic to healthy instances.
- Customizable protocols, paths, intervals and thresholds.

3 ROUTING STRATEGIES

- Round Robin
- Least Connections
- IP Hash
- Path/Host Based
- Consistent Hash
- Latency Based (Global)

4 TLS TERMINATION

- Offload SSL/TLS at the load balancer.
- Improves performance and reduces overhead on servers.
- Use secure policies and manage certificates centrally.

5 STICKY SESSIONS

- Maintain user session on the same backend.
- Use cookies or source IP hashing.
- Useful for stateful applications.

6 CONNECTION DRAINING

- Allow existing connections to finish before removing a target.
- Prevents dropped requests during deployments.
- Ensures zero-downtime updates.

7 WEIGHTED ROUTING

- Assign weights to targets or target groups.
- Route more traffic to higher capacity or newer versions.
- Ideal for canary and blue/green deployments.

8 FAILOVER

- Automatically reroute traffic from unhealthy or down targets.
- Multi-AZ and multi-region failover improve resilience.
- Use health checks and routing policies.

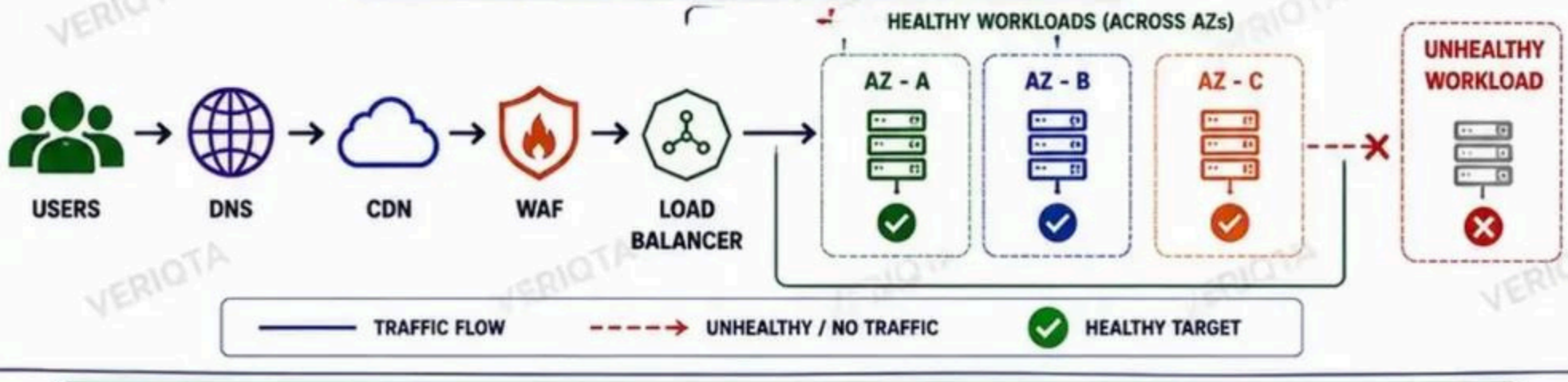
9 INTERNAL VS EXTERNAL LOAD BALANCERS

- External LB: Internet-facing.
- Internal LB: Private network access only.
- Choose based on security and access requirements.

10 BEST PRACTICES

- Use least-privilege security groups.
- Enable cross-zone load balancing.
- Monitor with metrics and logs.
- Design for failure and scale.

TRAFFIC DISTRIBUTION ARCHITECTURE



SENIOR ENGINEER TIP | **DESIGN THE TRAFFIC PATH FIRST. THEN CHOOSE THE RIGHT TOOLS AND CONFIGURATION.**

VERIQTA

DESIGNING THE CACHING ARCHITECTURE

CACHE SMART. REDUCE LOAD. IMPROVE SPEED. INCREASE RESILIENCE.

1 BROWSER CACHING
- Cache static assets in the user's browser.
- Use Cache-Control, ETag, and Expires headers.

2 CDN
- Cache content close to users globally.
- Reduce latency and origin server load.

3 REVERSE PROXY CACHING
- Cache at the edge or at the proxy layer.
- Examples: Nginx, Varnish, Cloudflare.

4 APPLICATION CACHING
- Cache frequent data within the application.
- Use in-memory caches (Local, Guava, etc.).

5 REDIS-STYLE DISTRIBUTED CACHE
- Centralized, fast key-value store.
- Share cache across all instances.
- Highly available with replication and clustering.

6 DATABASE QUERY CACHING
- Cache expensive or frequent queries.
- Reduce load on the database.

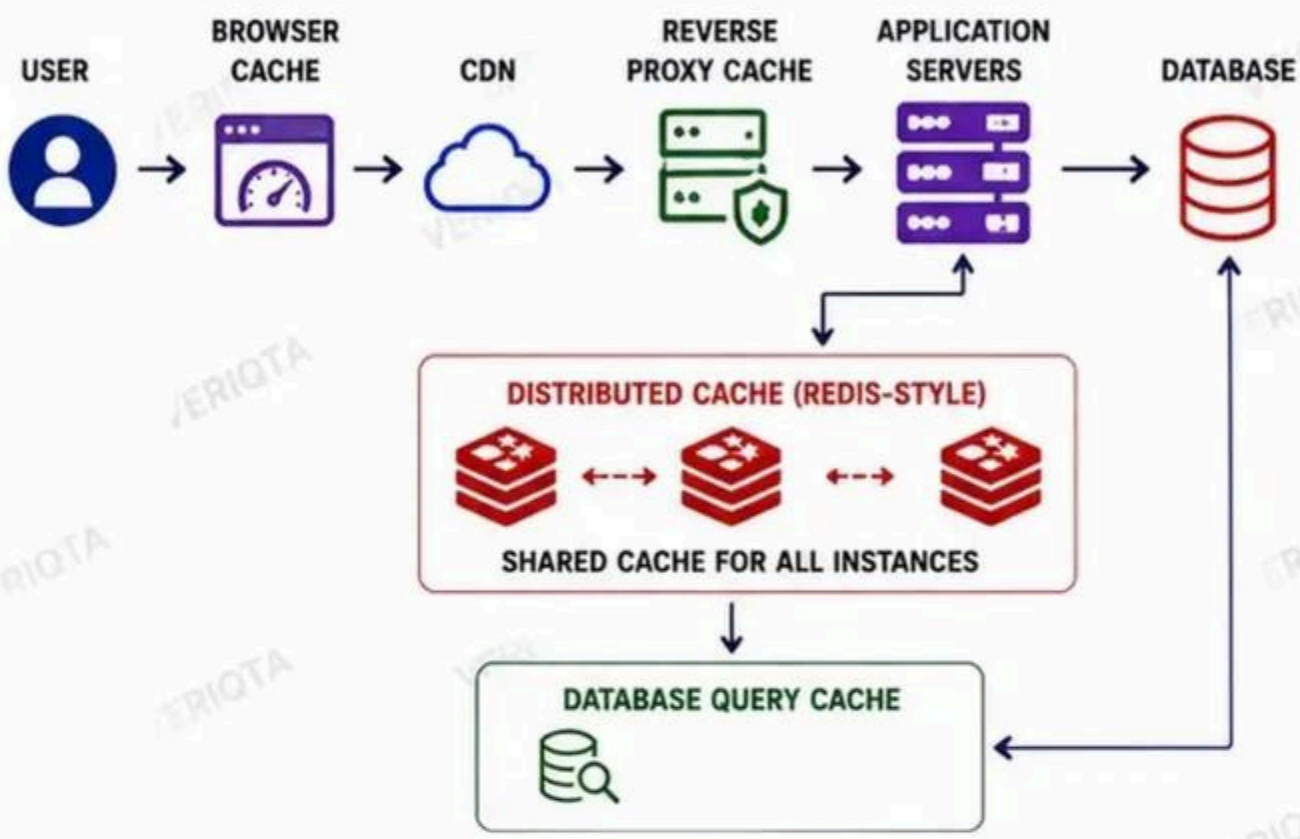
7 CACHE-ASIDE PATTERN
- Check cache first.
- If miss, read from DB, then store in cache.

8 TTL STRATEGY
- Set appropriate TTL for each type of data.
- Short TTL for dynamic data, long TTL for static data.

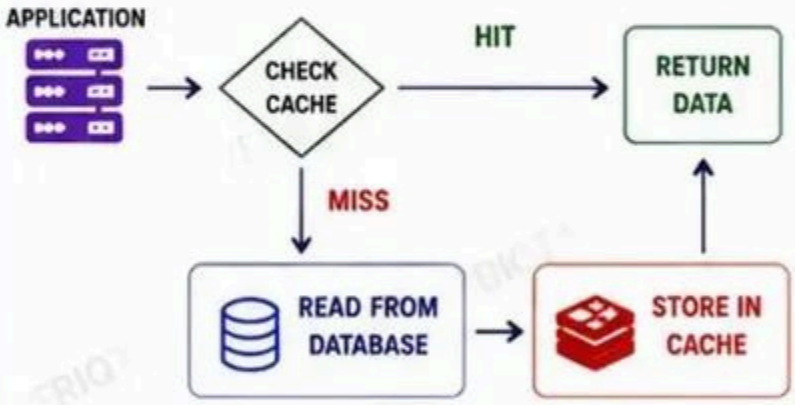
9 CACHE INVALIDATION
- Invalidate on updates.
- Use event-driven or time-based invalidation.

10 CACHE STAMPEDE
- Many requests hit the database when cache expires.
- Use locks, jittered TTL, or probabilistic early expiration.

MULTI-LAYER CACHING ARCHITECTURE



CACHE-ASIDE PATTERN (EXAMPLE FLOW)



BENEFITS
- Faster responses
- Reduced database load
- Better scalability
- Cost efficiency
- Improved availability

PRODUCTION FAILURE: WHAT HAPPENS WHEN THE CACHE CLUSTER FAILS?

CACHE CLUSTER DOWN
- Application cannot read/write to cache.
- All cache requests fail.
- Traffic goes directly to the database.
- Database load spikes dramatically.
- Increased latency and timeouts.
- Potential database failure.
- User experience degrades.

HOW TO MITIGATE
- Use cache client timeout + fallback.
- Implement circuit breaker.
- Graceful degradation of non-critical features.
- Use database query cache.
- Scale database for emergency.
- Monitor cache cluster health.

SENIOR ENGINEER TIP

CACHING IMPROVES PERFORMANCE,
BUT BAD CACHING INTRODUCES COMPLEXITY AND RISK.
DESIGN FOR CORRECTNESS FIRST, THEN ADD CACHE.

DATABASE ARCHITECTURE FOR DEVOPS ENGINEERS

SCALE READS. PROTECT WRITES. ENSURE HIGH AVAILABILITY.

1 PRIMARY AND REPLICA ARCHITECTURE
- One primary handles writes.
- Replicas handle reads.

2 READ SCALING
- Offload read traffic to replicas.
- Improve performance and reduce load on primary.

3 CONNECTION LIMITS
- Databases have max connections.
- Exceeding limits causes failures and timeouts.

4 CONNECTION POOLING
- Reuse connections efficiently.
- Reduce overhead and improve scalability.

5 REPLICATION LAG
- Replicas lag behind primary.
- Understand lag before serving critical reads.

6 FAILOVER
- Automatic or manual promotion of replica to primary.
- Test failover regularly.

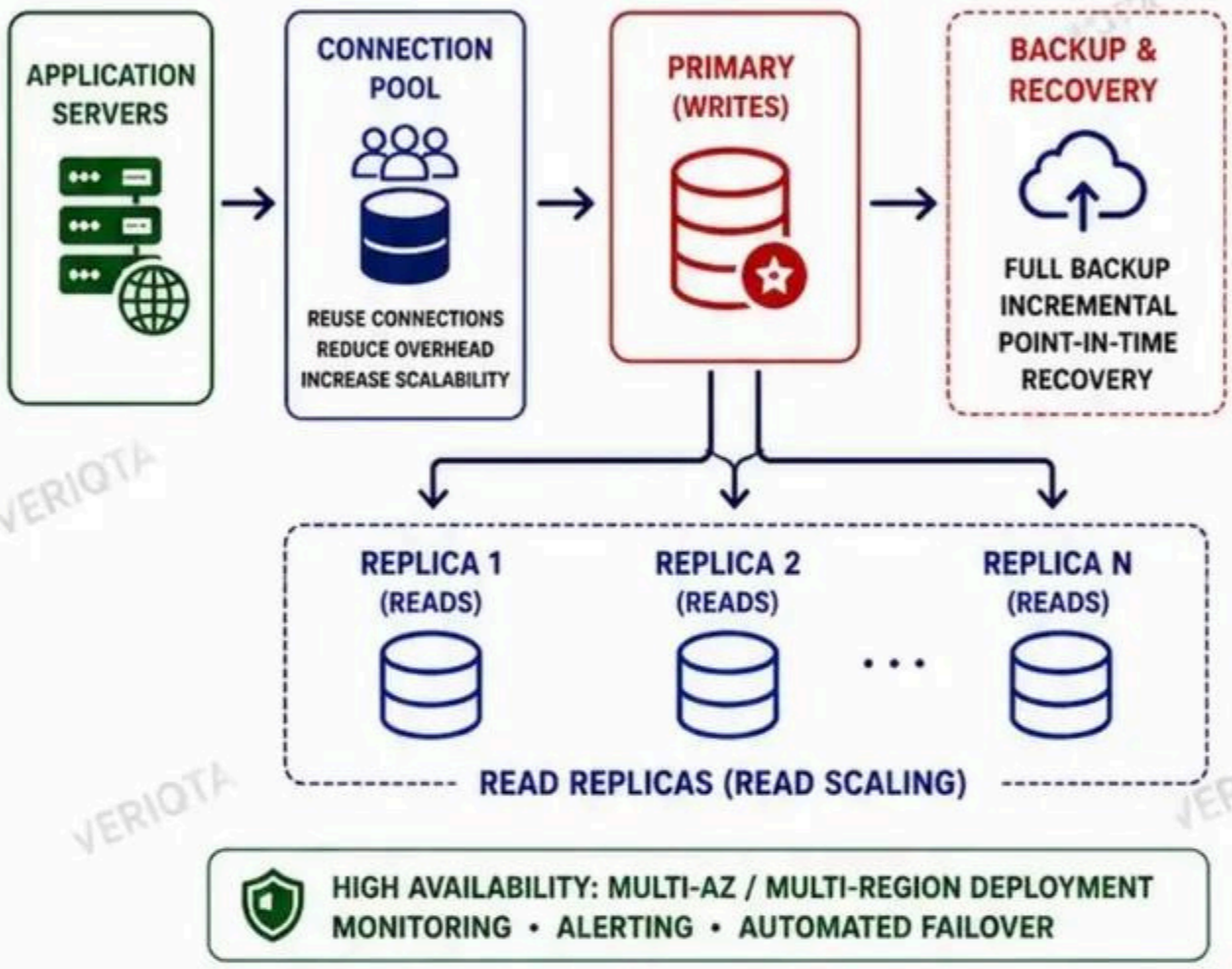
7 PARTITIONING
- Split large tables by range, list or hash.
- Improve manageability and query performance.

8 SHARDING
- Distribute data across multiple databases.
- Scale writes and storage horizontally.

9 BACKUP STRATEGY
- Automated backups.
- Point-in-time recovery.
- Test restores regularly.

10 DATABASE BOTTLENECKS
- Slow queries, locks, high CPU, disk I/O, or connection exhaustion.
- Monitor and optimize continuously.

TYPICAL DATABASE ARCHITECTURE



HIGH AVAILABILITY: MULTI-AZ / MULTI-REGION DEPLOYMENT
MONITORING • ALERTING • AUTOMATED FAILOVER

KEY CONSIDERATIONS

- ROUTE READS TO REPLICAS, ROUTE WRITES TO PRIMARY
- TEST FAILOVER AND RECOVERY REGULARLY
- UNDERSTAND REPLICATION LAG BEFORE SERVING CRITICAL READS
- MONITOR PERFORMANCE, QUERIES, AND METRICS
- USE CONNECTION POOLING TO AVOID CONNECTION EXHAUSTION
- PLAN FOR GROWTH: PARTITION OR SHARD WHEN NEEDED
- SECURE DATABASE ACCESS USING LEAST PRIVILEGE
- AUTOMATE BACKUPS AND DISASTER RECOVERY

COMMON DATABASE BOTTLENECKS

SLOW QUERIES | LOCK CONTENTION | HIGH CPU UTILIZATION | HIGH DISK I/O OR LATENCY | CONNECTION EXHAUSTION | REPLICATION LAG

DESIGN FOR SCALE. PREPARE FOR FAILURE. AUTOMATE EVERYTHING.

ARCHITECT ★ AUTOMATE ★ MONITOR ★ OPTIMIZE ★ RECOVER

VERIQTA

DESIGNING ASYNCHRONOUS SYSTEMS

DECOUPLE. SCALE. ABSORB PRESSURE. BUILD RESILIENT SYSTEMS.

1 SYNCHRONOUS VS ASYNCHRONOUS PROCESSING

- Synchronous: Request → Wait → Response
- Tight coupling, immediate result.
- Asynchronous: Request → Event → Processed Later
- Loose coupling, more resilient.

2 MESSAGE QUEUES

- Buffer messages between services.
- Decouple producers from consumers.
- Enable reliable, scalable communication.

3 EVENT-DRIVEN ARCHITECTURE

- Systems react to events.
- Real-time or near real-time processing.
- More scalable and responsive.

4 PRODUCERS AND CONSUMERS

- Producers send messages.
- Consumers process messages.
- Multiple consumers = parallel processing.

5 RETRY STRATEGY

- Retry transient failures with backoff.
- Use exponential backoff.
- Set retry limits to avoid infinite retries.

6 DEAD-LETTER QUEUES (DLQ)

- Store messages that cannot be processed.
- Prevent bad messages from blocking the queue.
- Monitor and fix the root cause.

7 IDEMPOTENCY

- Ensure an operation can be processed multiple times safely.
- Use unique IDs or deduplication keys.
- Critical for reliability.

8 DUPLICATE DELIVERY

- Messages may be delivered more than once.
- Design consumers to handle duplicates.
- Idempotency prevents side effects.

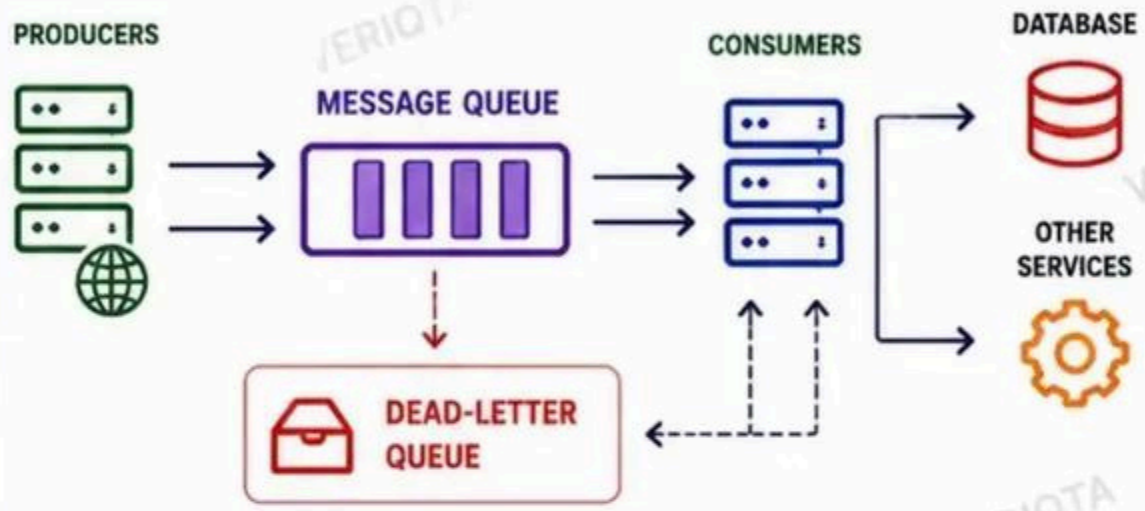
9 ORDERING

- queues do not guarantee order.
- Use ordering keys or partitions.
- Understand ordering trade-offs.

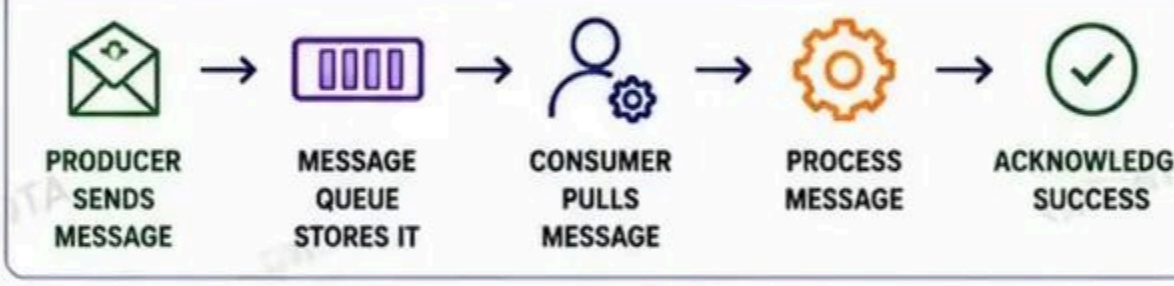
10 POISON MESSAGES

- Bad messages that always fail.
- Move to DLQ after max retries.
- Fix and reprocess safely.

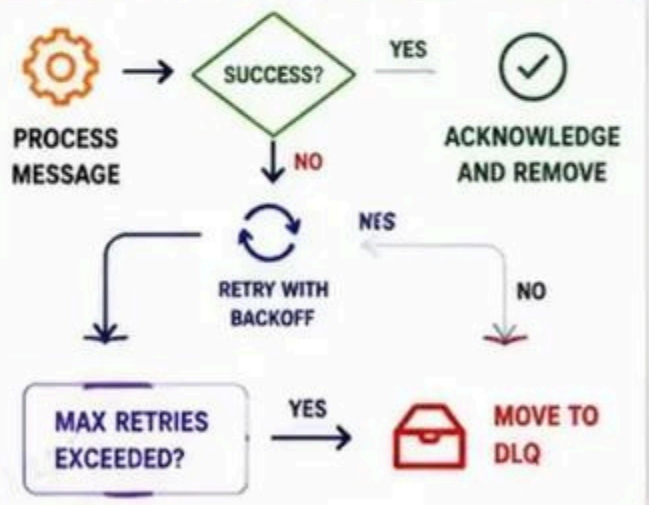
ASYNCHRONOUS ARCHITECTURE OVERVIEW



HOW MESSAGE FLOW WORKS



RETRY AND FAILURE HANDLING



BEST PRACTICES

- Keep messages small.
- Use meaningful message IDs.
- Make consumers idempotent.
- Handle failures gracefully.
- Monitor queue depth.
- Set proper visibility timeouts.
- Use DLQ and alerts.
- Test failure scenarios.

PRODUCTION NOTE

A QUEUE MOVES PRESSURE, IT DOES NOT REMOVE PRESSURE.

- Queues absorb spikes.
- Downstream systems can still be overwhelmed.
- Monitor backlog, not just processing rate.
- Design for backpressure and capacity limits.

COMMON USE CASES

ORDER PROCESSING | EMAIL NOTIFICATIONS | FILE PROCESSING | AUDIT LOGGING | INVENTORY UPDATES | DATA SYNC BETWEEN SERVICES

★ **BUILD LOOSELY COUPLED. DESIGN FOR FAILURE. DELIVER RELIABLY.** ★

KUBERNETES PLATFORM SYSTEM DESIGN

BUILD SCALABLE, RESILIENT, AND SELF-HEALING PLATFORMS

1 CONTROL PLANE BOUNDARIES

- Manage cluster state and scheduling.
- Keep it highly available and isolated.
- Do not run user workloads here.

2 WORKER NODES

- Run your containerized workloads.
- Scale nodes based on demand.
- Multiple AZs for high availability.

3 INGRESS

- Entry point for external traffic.
- TLS termination, routing, and rules.
- Use Ingress Controller (NGINX, ALB, etc.).

4 SERVICES

- Stable virtual IP for Pods.
- Enable service discovery and load balancing.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.

5 DEPLOYMENTS

- Declarative updates for Pods.
- Rolling updates and rollbacks.
- Ensure desired state and availability.

6 STATEFUL WORKLOADS

- Stable network identity and storage.
- Use StatefulSets for ordered creation and scaling.
- Ideal for databases, queues, etc.

7 PERSISTENT STORAGE

- Use PersistentVolume (PV) and PersistentVolumeClaim (PVC).
- Storage classes for dynamic provisioning.
- Data survives pod restarts.

8 HORIZONTAL POD AUTOSCALER

- Scale Pods based on CPU, memory, or custom metrics.
- Maintain performance automatically.
- Works with Deployments, StatefulSets.

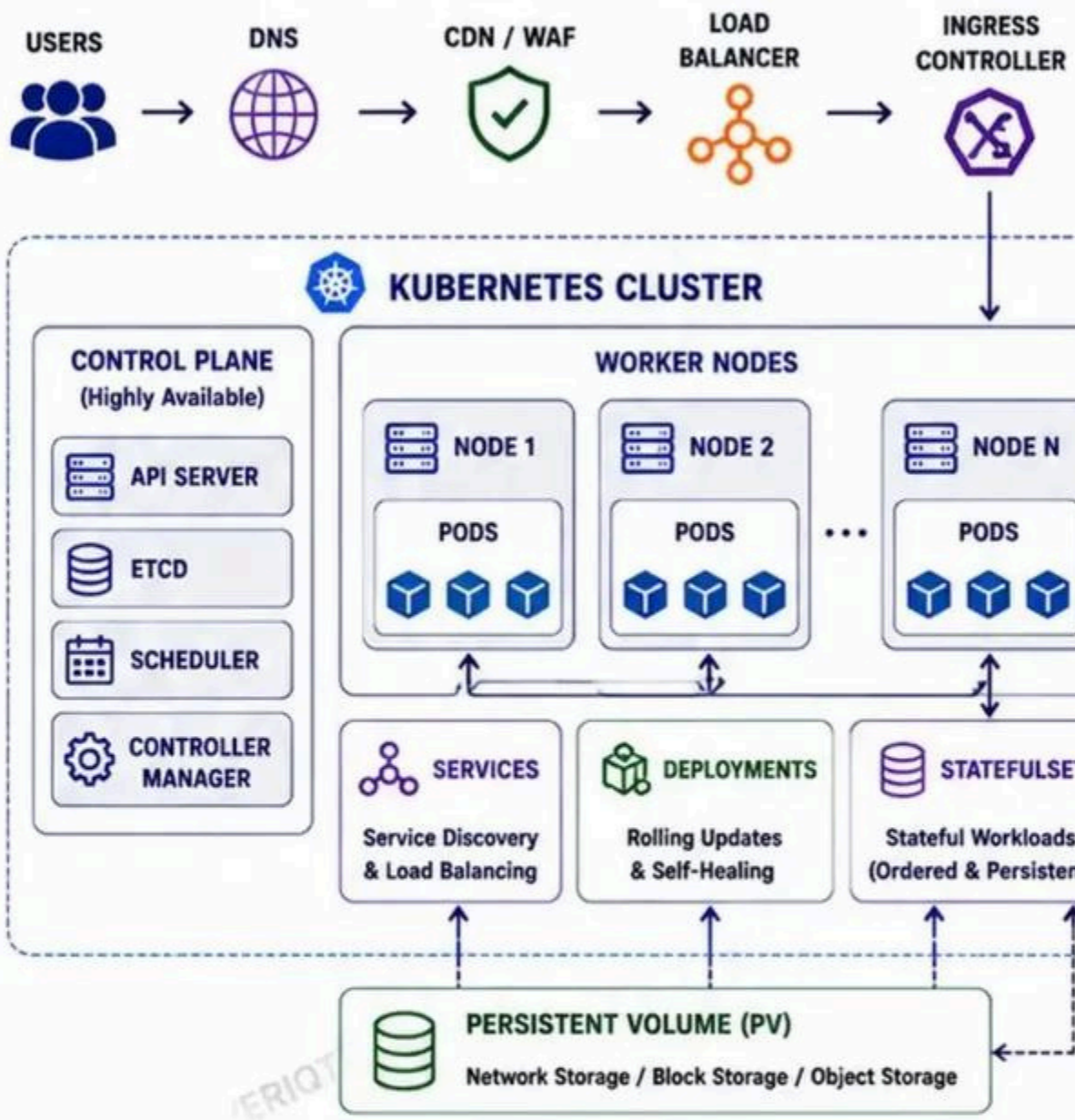
9 CLUSTER AUTOSCALER

- Automatically scale worker nodes.
- Adds nodes when Pods are pending.
- Removes nodes when unused.

10 PODDISRUPTIONBUDGET (PDB)

- Limit voluntary disruptions.
- Ensure minimum availability during updates, drains, and failures.
- Protects against downtime.

EXTERNAL TRAFFIC THROUGH THE KUBERNETES PLATFORM



AUTOSCALING & AVAILABILITY

HORIZONTAL POD AUTOSCALER (HPA)

Scales Pods based on metrics (CPU, Memory, Custom).

CLUSTER AUTOSCALER

Scales worker nodes in or out based on pending Pods.

POD DISRUPTION BUDGET (PDB)

Ensures minimum Pods are available during disruptions.

KEY BENEFITS

HIGH AVAILABILITY	SCALABILITY ON DEMAND	SELF-HEALING SYSTEMS	RESILIENCE BY DESIGN	EFFICIENT RESOURCE USE	EASY UPDATES & ROLLBACKS

DESIGN FOR FAILURE. AUTOMATE RECOVERY. OBSERVE EVERYTHING.

A WELL-DESIGNED KUBERNETES PLATFORM IS THE FOUNDATION OF RELIABLE SYSTEMS.

DESIGNING KUBERNETES FOR FAILURE

FAILURE WILL HAPPEN. DESIGN SO YOUR SYSTEM SURVIVES.

1 POD FAILURE

- Pods crash, OOM, or become unhealthy.
- Kubernetes restarts the pod based on RestartPolicy.

2 NODE FAILURE

- Entire node becomes unavailable.
- Pods are rescheduled on healthy nodes.

3 AVAILABILITY ZONE FAILURE

- AZ outage removes multiple nodes at once.
- Run replicas across multiple AZs.

4 REPLICA PLACEMENT

- Distribute replicas across nodes and zones.
- Avoid placing replicas on the same failure domain.

5 ANTI-AFFINITY

- Prevent multiple pods from running on the same node.
- Use podAntiAffinity rules.

6 TOPOLOGY SPREAD CONSTRAINTS

- Evenly spread pods across topology domains.
- Use topologySpreadConstraints for balance.

7 READINESS & LIVENESS PROBES

- Liveness: Is the container alive?
- Reaffiness: Is it ready to receive traffic?
- Unhealthy pods are restarted or removed from service.

8 RESOURCE REQUESTS & LIMITS

- Set requests for scheduling guarantees.
- Set limits to prevent noisy neighbor problems.

9 EVICTION

- Pods can be evicted due to resource pressure.
- Use requests/limits and monitoring to reduce evictions.

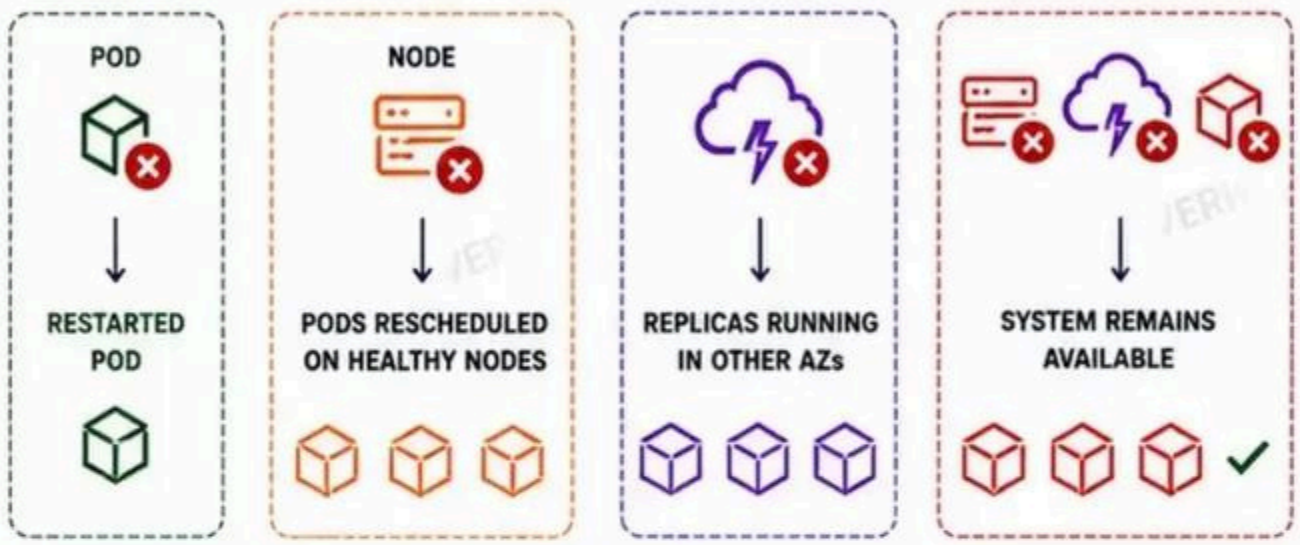
10 DISRUPTION BUDGETS (PDB)

- Ensure minimum replicas stay available during disruptions.
- Protects against voluntary and involuntary disruptions.

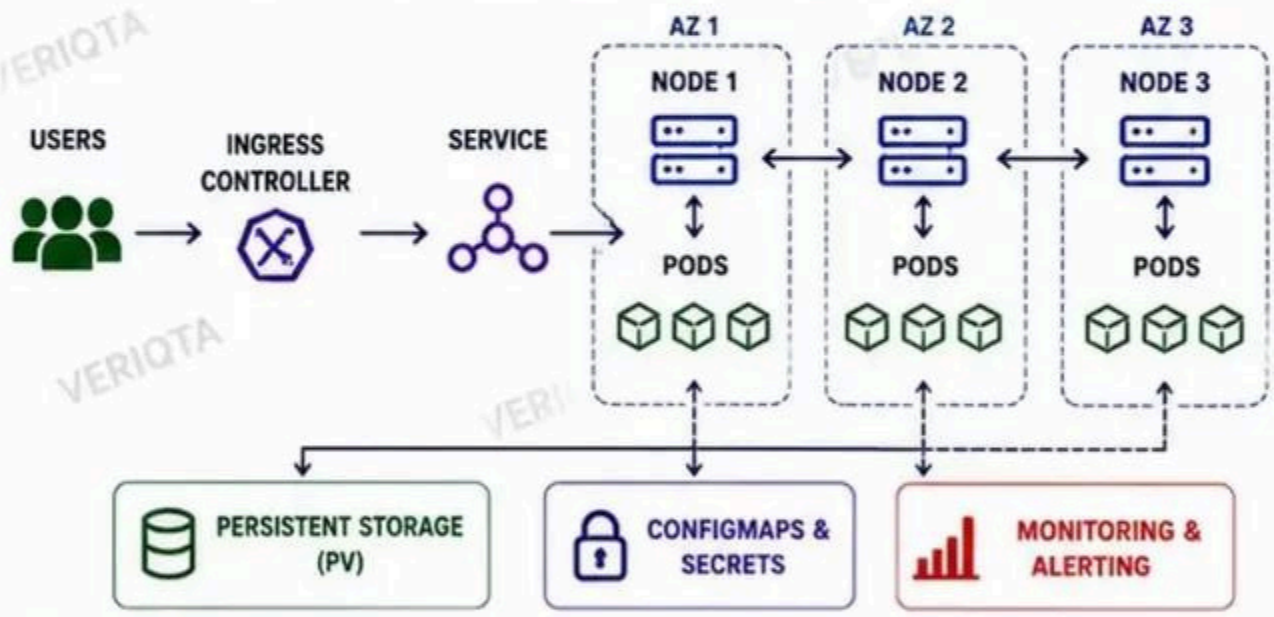
COMMON FAILURE CAUSES

- Single point of failure in architecture.
- Insufficient replicas or wrong placement.
- Ignoring AZ or node failures.
- No resource requests/limits.
- Missing probes or incorrect thresholds.
- No disruption budgets.
- Overloaded dependencies.

FAILURE SCENARIOS AND KUBERNETES RESPONSE



HIGH AVAILABILITY KUBERNETES DESIGN



RESILIENCY CONTROLS

MULTI-AZ DEPLOYMENT	HEALTH CHECKS	AUTO HEALING	SCALABILITY	SAFE UPDATES	DISRUPTION BUDGETS
Distribute workloads across AZs.	Use probes to detect unhealthy pods early.	Kubernetes restarts and reschedules failed pods.	HPA and Cluster Autoscaler handle growth.	Rolling updates with max unavailable controls.	Maintain minimum availability during disruptions.

PRODUCTION FAILURE:

HEALTHY CLUSTER, UNAVAILABLE APPLICATION

- All components report healthy.
- But users still experience errors or timeouts.
- Causes: dependency failure, saturation, misconfiguration, or cascading failure.
- Monitor the entire request path, not just pods.

DESIGN FOR FAILURE. VERIFY RESILIENCE. AUTOMATE RECOVERY.

★ BUILD SYSTEMS THAT STAY UP WHEN SOMETHING GOES DOWN. ★

VERIQTA

CI/CD SYSTEM DESIGN AT SCALE

AUTOMATE EVERYTHING. DELIVER SAFELY. SCALE CONTINUOUSLY.

1 SOURCE CONTROL
- Single source of truth.
- Branching strategy (main, develop, feature branches).

2 PULL-REQUEST VALIDATION
- Code review and approvals.
- Linting, static analysis.
- Unit tests on every PR.

3 BUILD WORKERS
- Ephemeral, scalable workers.
- Isolated build environments.
- Parallel builds for speed.

4 TEST STAGES
- Unit tests
- Integration tests
- End-to-end tests
- Smoke tests

5 SECURITY SCANNING
- SAST, DAST, dependency scan.
- Secrets scanning.
- Container image scanning.

6 ARTIFACT REPOSITORY
- Store build artifacts.
- Versioned and immutable.
- Promote through environments.

7 CONTAINER REGISTRY
- Store container images.
- Immutable tags.
- Vulnerability scan on push.

8 DEPLOYMENT CONTROLLER
- GitOps or pipeline-driven.
- Deploy to Kubernetes/VMs.
- Health checks and verification.

9 ENVIRONMENT PROMOTION
- Dev → Test → Staging → Prod
- Automated promotions.
- Consistent configurations.

10 APPROVAL BOUNDARIES
- Manual approvals for critical stages.
- Role-based access control.
- Audit everything.

11 ROLLBACK
- Automated rollback on failure.
- Quick recovery to last known good release.
- Blue/Green or Canary rollback.

CI/CD PIPELINE: COMMIT → PRODUCTION

ENVIRONMENTS FLOW

DEPLOYMENT STRATEGIES

- Rolling Deployment
- Blue/Green Deployment
- Canary Deployment
- Feature Flags
- Automated Rollback

PIPELINE SUPPORTING COMPONENTS

SECRETS MANAGEMENT | CONFIG MANAGEMENT | NOTIFICATIONS & ALERTS | OBSERVABILITY (LOGS, METRICS, TRACES) | AUDIT & COMPLIANCE | INFRASTRUCTURE AS CODE

PIPELINE BENEFITS

FASTER DELIVERY | HIGHER QUALITY | LOWER RISK | BETTER VISIBILITY | IMPROVED RELIABILITY

COMMON FAILURE POINTS

- BROKEN BUILDS
- FLAKY TESTS
- UNSCANNED VULNERABILITIES
- FAILED DEPLOYMENTS
- MISCONFIGURED ENVIRONMENTS
- MISSING APPROVALS
- SLOW ROLLBACKS

PRODUCTION NOTE:

A PIPELINE IS ONLY AS STRONG AS ITS TESTS, SECURITY, AND ROLLBACK STRATEGY.

- AUTOMATE, BUT NEVER REMOVE HUMAN OVERSIGHT WHERE RISK IS HIGH.
- VALIDATE EARLY, DEPLOY CONFIDENTLY.
- OBSERVE EVERYTHING, SO YOU CAN FIX ANYTHING.

★ COMMIT SMALL. TEST EVERYTHING. DEPLOY SAFELY. RECOVER QUICKLY. ★

VERIQTA

DESIGNING SAFE PRODUCTION DEPLOYMENTS

PROTECT USERS. REDUCE RISK. DEPLOY WITH CONFIDENCE.

1 ROLLING DEPLOYMENT

Gradually replace old instances with new ones.

2 BLUE/GREEN DEPLOYMENT

BLUE → GREEN

Deploy to the new environment and switch traffic when ready.

3 CANARY DEPLOYMENT

Release to a small subset of users before full rollout.

4 FEATURE FLAGS

Release code early, enable features safely.

5 TRAFFIC SHIFTING

Control the % of traffic going to new versions.

6 HEALTH VERIFICATION

Continuously check metrics, logs, and synthetic tests.

7 AUTOMATED ROLLBACK

Automatically roll back when health or metrics degrade.

8 DATABASE MIGRATION COMPATIBILITY

Ensure forward and backward compatibility of schema changes.

9 DEPLOYMENT OBSERVABILITY

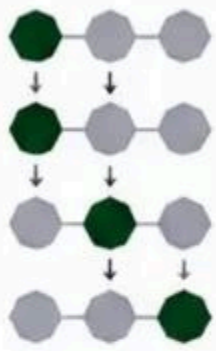
Track every deployment with metrics, logs, traces, and events.

10 BLAST-RADIUS CONTROL

Limit the impact of failures using isolation and boundaries.

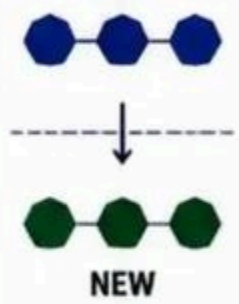
DEPLOYMENT STRATEGIES AT A GLANCE

ROLLING



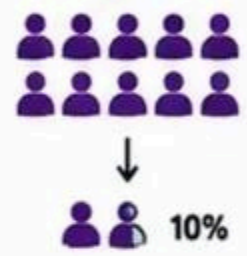
Update instances one at a time.

BLUE/GREEN



Switch from Blue to Green instantly.

CANARY



Test with 10% of users first.

TRAFFIC SHIFTING

Shift traffic 20% → 40% → 60% → 100%

DEPLOYMENT FLOW

1	2	3	4	5	6	7	8
CODE READY	BUILD & TEST	SECURITY SCAN	DEPLOY NEW VERSION	VERIFY HEALTH & METRICS	SHIFT TRAFFIC GRADUALLY	PROMOTE TO PRODUCTION	ROLLBACK IF NEEDED

AUTOMATED ROLLBACK CONDITIONS

- Error rate above threshold
- High latency
- 5xx response spike
- Health check failures
- Failed synthetic tests

DEPLOYMENT OBSERVABILITY

METRICS (RED, USE, SLI/SLO)	LOGS (APPLICATION & INFRA)	TRACES (DISTRIBUTED REQUESTS)	EVENTS (DEPLOYMENT TIMELINE)

BLAST-RADIUS CONTROL EXAMPLES

ENVIRONMENT ISOLATION	SERVICE BOUNDARIES	LIMITED TRAFFIC EXPOSURE	REGIONAL ISOLATION	PERMISSION BOUNDARIES

SENIOR ENGINEER TIP

DEPLOYMENT SUCCESS IS NOT THE SAME AS APPLICATION HEALTH.

- VERIFY REAL USER EXPERIENCE.
- LOOK AT GOLDEN SIGNALS.
- TRUST METRICS, NOT ASSUMPTIONS.
- OBSERVE BEFORE YOU CELEBRATE.

★ SAFE DEPLOYMENTS ARE DESIGNED. NOT HOPED FOR. ★

INFRASTRUCTURE AS CODE SYSTEM DESIGN

DEFINE INFRASTRUCTURE. VERSION IT. VALIDATE IT. DEPLOY IT. OPERATE IT.

1 REPOSITORY ARCHITECTURE
- Organize by purpose and environment.
- Clear structure and naming.
- Code, modules, and configs in Git.

2 MODULES
- Reusable and composable modules.
- Input variables and outputs.
- Keep modules small and focused.

3 ENVIRONMENT SEPARATION
- Separate dev, test, staging, prod.
- Separate accounts or subscriptions.
- Isolate data and networking.

4 REMOTE STATE
- Store state in remote backend.
- Shared and consistent state.
- Enable collaboration safely.

5 STATE LOCKING
- Prevent concurrent state changes.
- Use locking to avoid corruption.
- Ensure safe apply operations.

6 CI VALIDATION
- Lint, format, validate code.
- Security and compliance checks.
- Automated tests for modules.

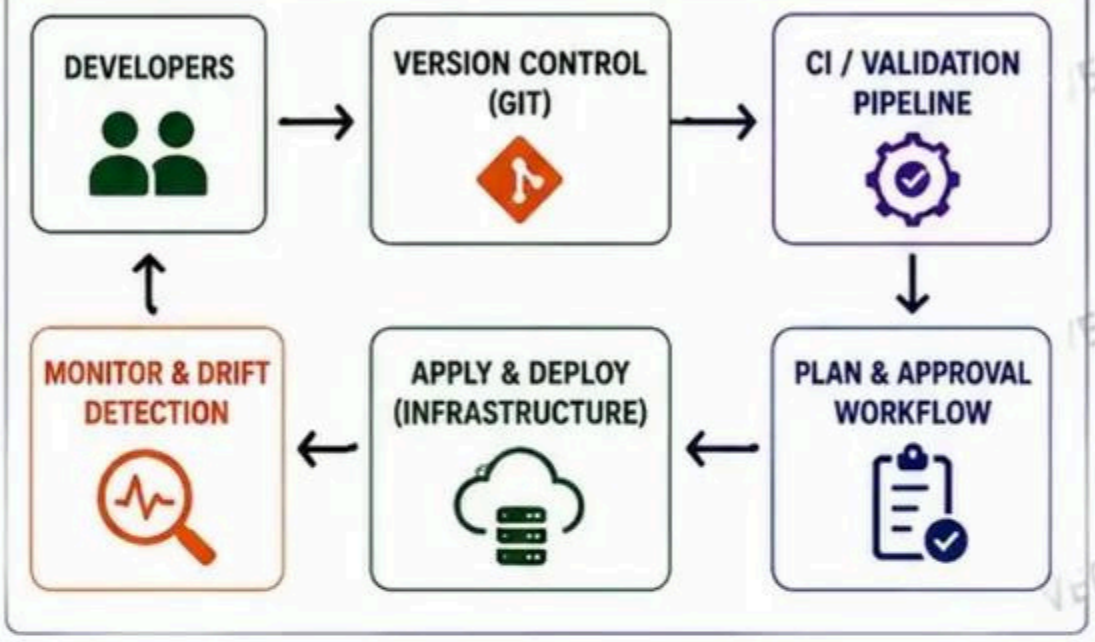
7 PLAN AND APPROVAL WORKFLOW
- Generate plan for every change.
- Review plan before apply.
- Approval required for critical envs.

8 DRIFT DETECTION
- Detect manual or external changes.
- Continuous drift monitoring.
- Alert and remediate drift.

9 SECRETS HANDLING
- Never store secrets in code.
- Use secret managers.
- Inject secrets securely at runtime.

10 PRODUCTION CHANGES
- Small, safe, and observable changes.
- Rollback strategy for every change.
- Monitor after every deployment.

IAC ARCHITECTURE OVERVIEW



GIT → VALIDATION → PLAN → APPROVAL → INFRASTRUCTURE

1. **COMMIT & PUSH** — Code changes pushed to Git
2. **VALIDATE** — Lint, security scan, unit tests, policy checks
3. **PLAN** — Generate execution plan and show diff
4. **APPROVAL** — Human review and approval for apply
5. **APPLY** — Apply changes to infrastructure safely

REPOSITORY STRUCTURE (EXAMPLE)

```
iac-repo/
├── modules/
│   ├── vpc/
│   ├── eks/
│   └── rds/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
├── shared/
├── scripts/
├── policies/
└── README.md
```

REMOTE STATE BACKEND (EXAMPLE)

- Object Storage (S3 / Blob)
- State Locking (DynamoDB / Table)
- Versioning Enabled
- Encryption Enabled
- Access Controlled
- Audit Logs Enabled

BEST PRACTICES

- ✓ Infrastructure is code.
- ✓ Every change is versioned.
- ✓ Every change is reviewed.
- ✓ Every change is validated.
- ✓ Every change is observable.
- ✓ Every change is reversible.
- ✓ Security is automated.
- ✓ Drift is detected early.
- ✓ Production is protected.

SENIOR ENGINEER TIP

DEPLOYMENT SUCCESS IS NOT THE SAME AS APPLICATION HEALTH.

- ★ DESIGN FOR SAFE CHANGE, NOT JUST FAST DEPLOYMENT.
- ★ AUTOMATE VALIDATION AND SECURITY.
- ★ MAKE DRIFT VISIBLE AND ACTIONABLE.
- ★ PROTECT PRODUCTION WITH APPROVALS AND GUARDRAILS.

★ **CODE YOUR INFRASTRUCTURE. OPERATE WITH CONFIDENCE.** ★

MULTI-ACCOUNT AND MULTI-ENVIRONMENT ARCHITECTURE

ISOLATE, ORGANIZE, AND GOVERN AT SCALE.

1 ENVIRONMENTS

- Development: Build and iterate.
- Testing: Validate functionality.
- Staging: Pre-production validation.
- Production: Live users and data.

2 ACCOUNT ISOLATION

- Separate AWS accounts per env.
- Limits blast radius.
- Strong security boundaries.
- Independent IAM and policies.

3 SHARED SERVICES

- Centralized services for all accounts.
- Examples: IAM, DNS, Logging, Security, CI/CD, Artifact Repo.
- Built once, used everywhere.

4 CENTRALIZED NETWORKING

- Hub-and-spoke or Transit Gateway.
- Shared networking services.
- Consistent routing and governance.
- Secure connectivity across accounts.

5 CENTRALIZED LOGGING

- Central log archive account.
- CloudTrail, Config, VPC Flow Logs.
- Cross-account log aggregation.
- Long-term retention and analysis.

6 IDENTITY BOUNDARIES

- Central identity management.
- Use IAM Identity Center (SSO).
- Least privilege across accounts.
- Clear role and permission mapping.

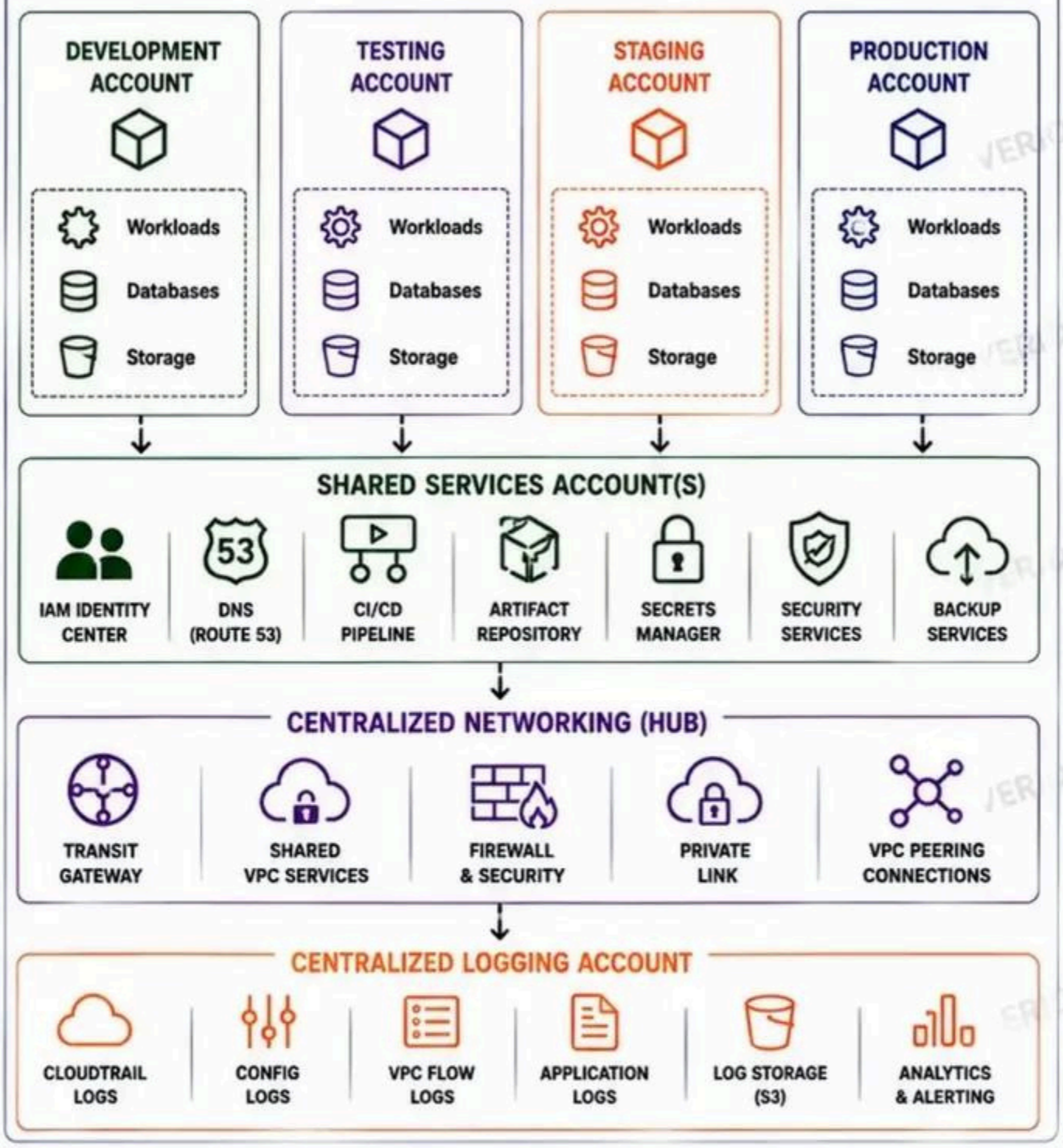
7 DEPLOYMENT BOUNDARIES

- Separate CI/CD access per account.
- Deployment roles per environment.
- Approval required for promotions.
- Prevent cross-env accidental deploys.

8 BLAST-RADIUS REDUCTION

- Isolate failures to one account.
- Restrict permissions.
- Limit network access.
- Protect data and workloads.

MULTI-ACCOUNT ARCHITECTURE OVERVIEW



DEPLOYMENT FLOW WITH BOUNDARIES

ENTERPRISE ADVICE

PRODUCTION ISOLATION IS A SECURITY AND RELIABILITY CONTROL.

- Isolation protects users and data.
- Boundaries enforce governance.
- Visibility and logging drive confidence.
- Least privilege reduces risk.

CLEAR BOUNDARIES. SHARED SERVICES. CENTRALIZED VISIBILITY. REDUCED RISK.

VERIQTA

OBSERVABILITY SYSTEM DESIGN

YOU CAN'T IMPROVE WHAT YOU CAN'T OBSERVE.

1 METRICS
- Numeric measurements over time.
- System, application, and business metrics.
- Time series data.
- Example: CPU, latency, errors.

2 LOGS
- Discrete events with context.
- For debugging and audit trails.
- Structured logging is essential.
- Example: Error logs, access logs.

3 TRACES
- Follow a request across services.
- Visualize latency and dependencies.
- Trace ID, Span ID, and timing.
- Example: End-to-end request flow.

4 OPENTELEMETRY
- Vendor-neutral instrumentation.
- Unified collection for metrics, logs, and traces.
- Standardized semantic conventions.

5 COLLECTION ARCHITECTURE
- Agents/Sidecars collect telemetry.
- Collectors receive, process, and export.
- Scalable, reliable, and resilient.

6 CENTRALIZED STORAGE
- Store large volumes of telemetry.
- Optimized for scale and retention.
- Separate storage per data type.
- Long-term + short-term tiers.

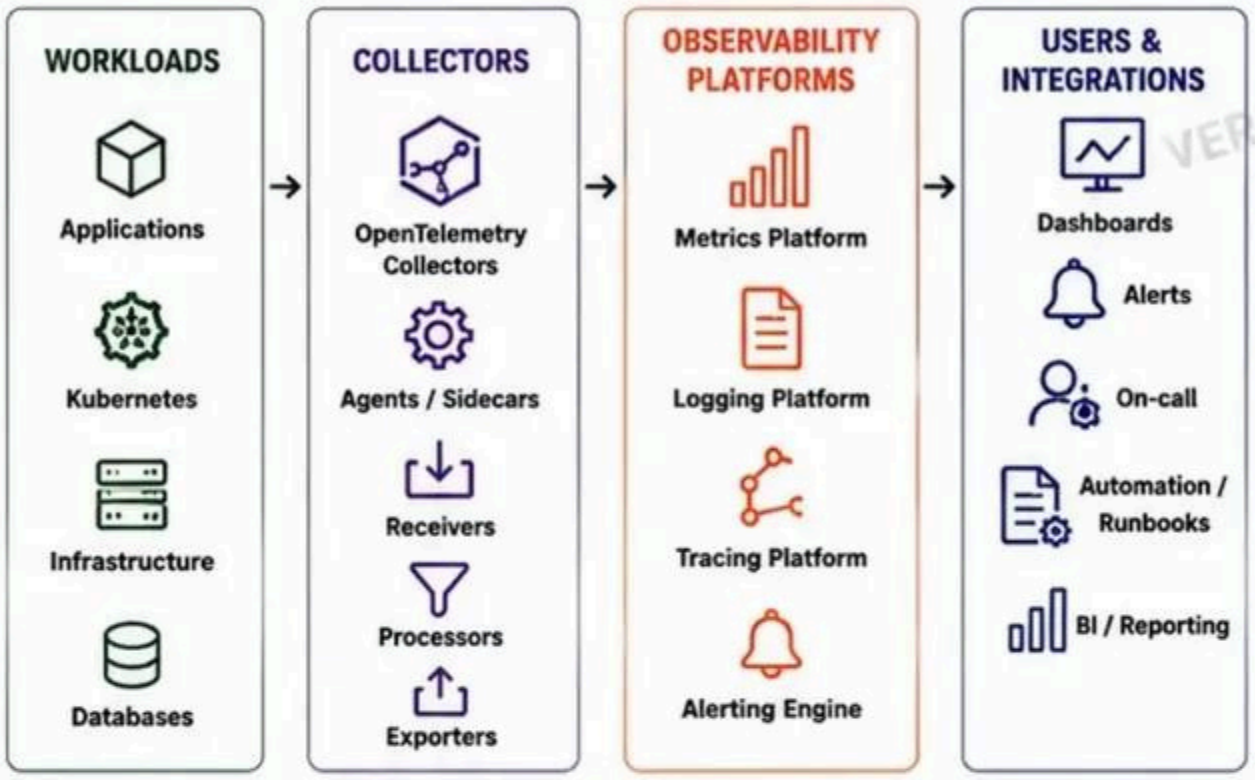
7 SERVICE DASHBOARDS
- Per-service visibility.
- Golden signals and RED/USE.
- Custom business dashboards.
- Actionable and real-time.

8 SLI AND SLO DESIGN
- Define good service level (SLI).
- Set objectives (SLO).
- Measure error budgets.
- Drive reliability and priorities.

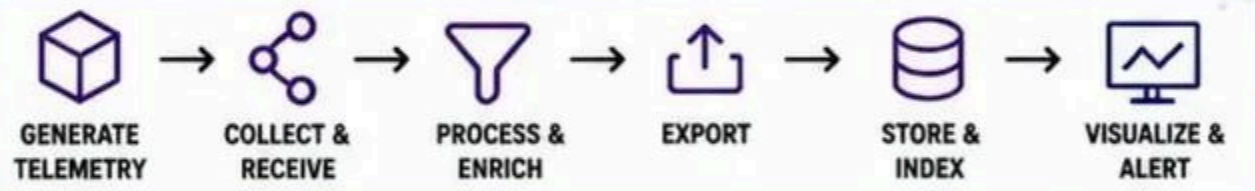
9 ALERT ROUTING
- Route alerts to the right people.
- Severity-based routing.
- On-call schedules and escalation.
- Reduce noise and alert fatigue.

10 HIGH-CARDINALITY TELEMETRY
- Unique dimensions can explode costs.
- Control cardinality at the source.
- Use labels wisely.
- Monitor, sample, or aggregate.

OBSERVABILITY ARCHITECTURE OVERVIEW



DATA FLOW: WORKLOADS → COLLECTORS → OBSERVABILITY PLATFORMS



GOLDEN SIGNALS

- **LATENCY** — How long requests take
- **TRAFFIC** — How many requests
- **ERRORS** — Rate of failed requests
- **SATURATION** — How busy the system is

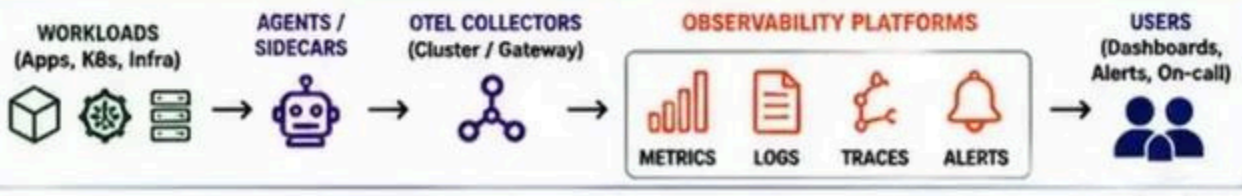
TELEMETRY TYPES

- **METRICS** — Numerical time-series data
- **LOGS** — Discrete events with context
- **TRACES** — Request flow across services
- **PROFILES** — CPU, memory, and runtime profiling

BEST PRACTICES

- Instrument everything from the start.
- Use consistent naming and tagging.
- Create actionable dashboards.
- Alert on symptoms, not just infrastructure.
- Continuously review SLIs and alerts.
- Control cardinality and data volume.

EXAMPLE COLLECTION ARCHITECTURE



SENIOR ENGINEER TIP

DEPLOYMENT SUCCESS IS NOT THE SAME AS APPLICATION HEALTH.

- OBSERVE WHAT USERS EXPERIENCE.
- CREATE SLOS THAT MATCH BUSINESS OUTCOMES.
- USE TELEMETRY TO UNDERSTAND, NOT JUST TO ALERT.
- HIGH QUALITY TELEMETRY = FASTER INCIDENT RESOLUTION.

★ GOOD OBSERVABILITY IS BUILT. NOT BOUGHT. ★

VERIQTA

DESIGNING RELIABLE INCIDENT DETECTION

DETECT EARLY. ALERT SMART. RESPOND FASTER.

1 SYMPTOMS VS CAUSES

- Symptoms show what is wrong.
- Causes explain why it is wrong.
- Alerts should point to causes, not just symptoms.

2 GOLDEN SIGNALS

- Latency – How long requests take.
- Traffic – How many requests.
- Errors – How many are failing.
- Saturation – How full the system is.

3 ALERT THRESHOLDS

- Set based on user impact.
- Use dynamic baselines.
- Avoid static thresholds for dynamic systems.

4 BURN-RATE ALERTS

- Detect issues fast and predict future outages.
- Multi-window, multi-burn-rate approach.

5 DEPENDENCY MONITORING

- Monitor external services.
- Track latency, errors, and availability.
- Know when dependencies fail you.

6 SYNTHETIC MONITORING

- Test from user locations.
- Validate critical journeys.
- Detect issues before users report.

7 ALERT DEDUPLICATION

- Group similar alerts.
- Reduce noise and duplicates.
- One incident, one alert.

8 ALERT FATIGUE

- Too many alerts = ignored alerts.
- Focus on actionable alerts.
- Quality over quantity.

9 ESCALATION

- Route to the right people.
- Escalate based on urgency.
- Clear ownership and follow-up.

10 RUNBOOK INTEGRATION

- Alerts should guide responders.
- Link alerts to runbooks.
- Speed up diagnosis and resolution.

GOLDEN SIGNALS – THE FOUNDATION

LATENCY	TRAFFIC	ERRORS	SATURATION
How long requests take.	How many requests.	How many requests fail.	How full the system resources are.
High latency = poor user experience.	Sudden drops or spikes matter.	Errors impact reliability.	High saturation leads to failure.

ALERTING DESIGN PRINCIPLES

FOCUS ON USER IMPACT	REDUCE NOISE	ALERT EARLY, NOT LATE	MAKE ALERTS ACTIONABLE	ROUTE TO THE RIGHT PEOPLE	LINK ALERTS TO RUNBOOKS

INCIDENT DETECTION ARCHITECTURE

SOURCES	COLLECT	PROCESS	ANALYZE	ALERT
Applications	Metrics	Aggregate	Thresholds	Alert Manager
Infrastructure	Logs	Normalize	Anomaly Detection	Routing
Databases	Traces	Correlate	Burn-rate	Escalation
Services	Events	Enrich	Dependency	Notifications
User Journeys	Synthetic Checks	Deduplicate	Analysis	Runbooks

ALERT MATURITY MODEL

LEVEL 1 REACTIVE	Many alerts, high noise, no clear ownership.
LEVEL 2 BASIC	Threshold-based alerts, manual routing.
LEVEL 3 STANDARDIZED	Golden signals, routing, runbooks, basic SLOs.
LEVEL 4 PROACTIVE	Anomaly detection, burn-rate, SLO alerts.
LEVEL 5 AUTONOMOUS	AI-assisted, auto-remediation, continuous improvement.

COMMON PITFALLS

- ALERT ON INFRASTRUCTURE, NOT USER IMPACT.
- TOO MANY ALERTS.
- POOR THRESHOLD SETTINGS.
- NO CORRELATION OR DEDUPLICATION.
- MISSING DEPENDENCY VISIBILITY.
- NO RUNBOOK OR UNCLEAR ACTIONS.
- NO POST-INCIDENT LEARNING.

DEBUGGING INSIGHT

MONITOR WHAT USERS EXPERIENCE, NOT ONLY INFRASTRUCTURE HEALTH.

- START FROM THE USER.
- FOLLOW THE REQUEST.
- COMBINE METRICS, LOGS, AND TRACES.
- FIND THE CAUSE, NOT JUST THE ALERT.

★ YOU CAN'T FIX WHAT YOU CAN'T SEE. OBSERVE. ALERT. ACT. ★

SECRETS, IDENTITY, AND SECURITY ARCHITECTURE

SECURE BY DESIGN. MINIMIZE RISK. PROTECT EVERY SECRET.

1 HUMAN VS WORKLOAD IDENTITY

- Human identity: users, admins, engineers.
- Workload identity: applications, services, containers.
- Use the right identity for the right purpose.
- Never use human credentials for workloads.

2 SHORT-LIVED CREDENTIALS

- Use temporary credentials whenever possible.
- Reduce the impact of leaked credentials.
- Use STS, OIDC, or workload identity federation.
- Rotate and expire automatically.

3 IAM ROLES

- Use IAM roles for applications and services.
- Avoid long-term access keys.
- Use trust policies and conditions carefully.
- Separate roles by responsibility.

4 LEAST PRIVILEGE

- Grant only the minimum permissions needed.
- Avoid admin and wildcard policies.
- Review and refine permissions regularly.
- Least privilege is a continuous process.

5 SECRET STORES

- Store secrets in managed secret stores.
- Use AWS Secrets Manager or Parameter Store.
- Never hardcode secrets.
- Control access with IAM policies.

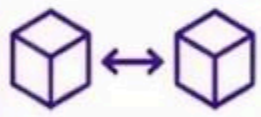
6 SECRET ROTATION

- Rotate secrets automatically.
- Reduce exposure from compromised credentials.
- Use managed rotation whenever possible.
- Monitor rotation success.

7 ENCRYPTION

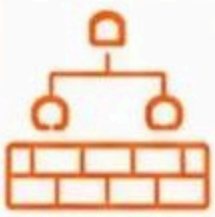
- Encrypt data in transit (TLS).
- Encrypt data at rest.
- Use KMS for key management.
- Use customer-managed keys for critical workloads.

8 SERVICE-TO-SERVICE AUTHENTICATION



- Authenticate services, not IPs.
- Use mutual TLS or signed tokens.
- Validate identity and authorization on every request.
- Avoid shared secrets between services.

9 NETWORK SEGMENTATION



- Segment networks by trust level.
- Use private subnets for workloads.
- Restrict traffic with security groups, NACLs, and firewall rules.
- Assume breach and isolate.

10 AUDIT TRAILS

- Enable CloudTrail for all accounts.
- Monitor API activity.
- Log access to secrets and keys.
- Centralize and protect audit logs.
- Review logs continuously.

SECURITY REMINDER

CREDENTIALS SHOULD NOT TRAVEL THROUGH THE CI/CD SYSTEM UNNECESSARILY.

- PASS SECRETS DIRECTLY TO THE DESTINATION.
- USE OIDC OR WORKLOAD IDENTITY.
- AVOID STORING SECRETS IN PIPELINES.
- MINIMIZE EXPOSURE. MAXIMIZE SECURITY.

★ STRONG IDENTITY. PROTECTED SECRETS. SECURE SYSTEMS. ★